Nakijk 02_02 — V3

NAKIJK V3 · Hoofdstuk 02 · Volledige CRUD · bij Oefening_02_02.md · 16 oktober 2026

Nagekeken: `ProgrammingAdvancedOefeningen` → `Models/Ventilator.cs` (11 oktober) en `Controllers/VentilatorController.cs` (16 oktober, 17:53).

> **OPMERKING**
>
> Sinds V2 is je lijst `private static` en heet ze `ventilatoren`. Een nieuwe ventilator blijft nu bewaard. De twee overbodige `using`-regels zijn ook weg.
>
> Je vroeg hoe je aan punt 4 begint: een veld dat leeg of fout is, weigeren met 400. Dat staat in een eigen hoofdstuk, **Zo begin je aan punt 4**.
>
> In je bestand staan al een PUT en een DELETE. Zoals je vroeg, kijk ik die ook volledig na.
>
> Ik kon je API niet starten, want op deze werkruimte staat geen .NET. Ik heb je code gelezen en elk verzoek op papier nagelopen.

In één oogopslag

Het bewaren werkt nu. Je bent aan de controles van punt 4 begonnen, maar je eerste controle doet nog niets: een leeg merk geeft nog altijd 201 (A1). In je DELETE zit dezelfde fout, en daar waarschuwt de compiler je ook voor (A2). Verder wijken drie berichten een letter of een woord af van de opgave (A3).

Punt	Wat de opgave vraagt	Status
1	het model `Ventilator`	✓ in orde
2	de controller, met ventilatoren die bewaard blijven	✓ opgelost sinds V2
3	`GET api/ventilator` en `GET api/ventilator/{id}`	½ zie A3
4	`POST api/ventilator` met 201 en het adres	✓ in orde
4	de drie controles met 400	✗ begonnen, zie A1 en Zo begin je aan punt 4
5	`PUT api/ventilator/{id}`	½ zie A3; de controles komen nog
6	`DELETE api/ventilator/{id}`	✗ zie A2 en A3
7	de twee testen	✗ volgt uit A1 en A2
8	`PUT api/ventilator/vervang/{id}`	nog te doen

Zo antwoordt je API nu:

Verzoek	Je API antwoordt nu	De opgave vraagt
`GET api/ventilator`	200 met de vier ventilatoren	✓
`GET api/ventilator/99`	404 met *We vonden geen ventilator met Id 99*	hetzelfde, met een punt achteraan
`POST api/ventilator` met een geldige ventilator	201, Id 5, adres *api/Ventilator/5*	✓
daarna `GET api/ventilator`	vijf ventilatoren	✓
`POST` met `"merk": ""`	201, en hij staat in de lijst	400 met *Een ventilator moet een merk hebben.*
`POST` met 92 mm	201, en hij staat in de lijst	400 met *We verkopen enkel ventilatoren van 120 of 140 mm.*
`POST` met prijs 0	201, en hij staat in de lijst	400 met *De prijs moet groter zijn dan 0.*
`PUT api/ventilator/1` met geldige gegevens	204	✓
`PUT api/ventilator/99`	404 met *Kan geen Ventilator bijwerken met Id 99, omdat deze niet bestaat.*	hetzelfde, met een kleine v
`PUT api/ventilator/1` met 92 mm	204, en ventilator 1 heeft nu 92 mm	400
`DELETE api/ventilator/2`	204	✓
nog eens `DELETE api/ventilator/2`	204	404 met *Kan ventilator met Id 2 niet verwijderen, omdat deze niet gevonden is.*
`PUT api/ventilator/vervang/1`	404, zonder bericht	201 met de nieuwe ventilator

A. Wat een verkeerd antwoord geeft

A1. Je controle op het merk geeft nooit een 400

Waar: `CreateVentilator`, regel 40 tot 43.

Verzoek	Je API antwoordt nu	De opgave vraagt
`POST` met `"merk": ""`	201, en de ventilator zonder merk staat in de lijst	400 met *Een ventilator moet een merk hebben.*
`POST` met `"merk": "  "`	201	400 met hetzelfde bericht
`POST` met `"merk": null`	400, maar met een Engelse melding van ASP.NET, niet de jouwe	(zie stap 3 hieronder)

Er zijn twee redenen, en ze staan los van elkaar.

1. Het antwoord wordt gemaakt, maar niet teruggestuurd. `BadRequest(...)` op regel 42 maakt een antwoord met statuscode 400. Daar stopt het: er gebeurt niets mee. Het antwoord verdwijnt, je methode gaat gewoon verder naar regel 45 en 47, en de client krijgt 201.

Kijk naar je eigen `GetVentilatorById`. Op regel 28 doe je het wel goed. Wat staat daar vóór `NotFound`, dat op regel 42 vóór `BadRequest` ontbreekt?

2. `null` komt nooit tot op regel 40. `Merk` is een `string` zonder vraagteken, en je project staat op `Nullable` aan. ASP.NET behandelt het merk dan als verplicht, en `[ApiController]` weigert een `"merk": null` zelf, nog voor je methode start. Wat wél tot bij je methode komt, is een lege tekst `""` of een tekst met enkel spaties. Je controle kijkt alleen naar `null`, en laat die twee dus door.

De compiler zegt hier niets. Een methode oproepen zonder iets met het resultaat te doen, is toegelaten. Dat maakt deze fout verraderlijk: alles compileert, en toch gebeurt er niets.

Richting: zie **Zo begin je aan punt 4**, stap 2 en stap 3.

A2. Een DELETE op een onbekend Id geeft 204

Waar: `DeleteVentilator`, regel 73 tot 78.

Verzoek	Je API antwoordt nu	De opgave vraagt
`DELETE api/ventilator/2`	204	✓
nog eens `DELETE api/ventilator/2`	204	404 met *Kan ventilator met Id 2 niet verwijderen, omdat deze niet gevonden is.*
`DELETE api/ventilator/99`	204	404 met het bericht hierboven

Waarom. Dezelfde fout als in A1, reden 1. `NotFound(...)` op regel 75 maakt een 404, maar stuurt ze niet terug. Je methode gaat verder naar regel 78 en roept `Remove` op met `null`. Een `List` zoekt dan naar een lege plaats, vindt er geen en geeft `false` terug: er crasht niets, er wordt niets verwijderd. Daarna geeft regel 80 een 204.

Hier waarschuwt de compiler je wel, op regel 78:

CS8604, Possible null reference argument for parameter 'item' in 'bool List<Ventilator>.Remove(Ventilator item)'.

De compiler ziet dat `ventilator` op regel 78 nog altijd `null` kan zijn. Dat is het teken dat je `if` de methode niet stopt.

Richting. Zelfde oplossing als in A1, reden 1. Een goede controle voor jezelf: zodra je `if` de methode echt stopt, weet de compiler dat `ventilator` op regel 78 nooit `null` is, en verdwijnt de waarschuwing vanzelf.

A3. Drie berichten die net afwijken

Een mens leest erover, maar het nakijkscript vergelijkt elk bericht letterlijk.

Waar	Jouw bericht	De opgave
regel 28, GET	We vonden geen ventilator met Id {id}	We vonden geen ventilator met Id {id}**.**
regel 57, PUT	Kan geen **V**entilator bijwerken met Id {id}, omdat deze niet bestaat.	Kan geen **v**entilator bijwerken met Id {id}, omdat deze niet bestaat.
regel 75, DELETE	…, omdat deze niet **is gevonden**.	…, omdat deze niet **gevonden is**.

Richting. Kopieer elk bericht uit de opgave en plak het in je code. Dan sluipt er geen tikfout in.

Nog te doen

Punt	Wat er nog moet komen	Waar je de richting vindt
4	de controle op de afmeting en op de prijs, en de controle op het merk afwerken	**Zo begin je aan punt 4**
5	dezelfde drie controles in je PUT	stap 8 van **Zo begin je aan punt 4**
7	de twee testen: 92 mm geeft 400, een tweede DELETE geeft 404	werkt zodra A1, A2 en punt 4 af zijn
8	`PUT api/ventilator/vervang/{id}`	V1, A6

Bij je PUT: hij zoekt nu eerst en geeft een 404 als de ventilator niet bestaat. Zet de controles daarna, vóór je de gegevens overneemt. Bestaat de ventilator niet, dan is dat het eerste wat de client moet weten. En wat je overneemt en daarna afkeurt, staat al in je lijst.

Zo begin je aan punt 4

In les 02 staat `BadRequest` in de tabel met statuscodes, maar de les toont niet hoe je een veld controleert. Je kent het patroon wel al. In `GetVentilatorById` stel je een vraag (staat er een ventilator met dit Id?) en geef je een antwoord dat de methode stopt (een 404). Een 400 werkt precies zo. Alleen de vraag en de statuscode zijn anders.

Stap 1. Leg de volgorde van je methode vast

Schrijf de stappen eerst in gewone zinnen, op papier of als commentaar in je methode:

1. Is het merk leeg? Dan 400 met het bericht, en stop.
2. Is de afmeting niet toegelaten? Dan 400 met het bericht, en stop.
3. Is de prijs te laag? Dan 400 met het bericht, en stop.
4. Pas als je hier geraakt: het Id bepalen, toevoegen en 201 teruggeven.

Eerst controleren, dan pas iets doen. Wat je toevoegt en daarna afkeurt, staat al in de lijst.

Je bepaalt het Id nu vóór de controle, op regel 37 en 38. Dat doet geen kwaad, want het object komt bij een 400 niet in de lijst. Maar het leest vreemd: je geeft een nummer aan iets dat je misschien weigert. Zet het Id bij stap 4.

Stap 2. Elke controle eindigt met `return`

`BadRequest`, `NotFound`, `Ok` en `CreatedAtAction` **maken** een antwoord. Ze **sturen** het niet. Pas `return` stuurt het antwoord naar de client, en stopt meteen je methode. Wat eronder staat, wordt dan niet meer uitgevoerd. Dat is precies wat je wil: een geweigerde ventilator komt zo nooit bij `Add`.

Zo controleer je jezelf. Lees je methode regel per regel, zoals de computer dat doet, met een leeg merk in je hoofd. Waar stopt hij? Komt hij voorbij `Add`, dan klopt er iets niet.

In je leerboek, hoofdstuk 26, staan alle helpers met hun statuscode. Daar staat ook een voorbeeld van een `BadRequest` met een bericht.

Stap 3. Wat is een leeg merk?

Wat de client stuurt	Komt het tot in je methode?	Moet het een 400 geven?
`"merk": "Noctua"`	ja	nee
`"merk": ""`	ja	ja
`"merk": "  "`	ja	ja: wie enkel spaties typt, heeft niets ingevuld
geen merk in de body	ja, met de lege beginwaarde uit je model	ja
`"merk": null`	nee, ASP.NET weigert het zelf, met een Engelse melding	daar heb je geen invloed op

Richting. Je leerboek, hoofdstuk 1, heeft een tabel *Hoe lang is het, is het leeg?*. Daar staat één methode die `null`, een lege tekst én enkel spaties in één keer nagaat. Een vergelijking met `null` alleen, of met `""` alleen, is niet genoeg.

Stap 4. 120 of 140: twee waarden, geen bereik

- Het zijn **twee toegelaten waarden**, geen bereik. Een controle "tussen 120 en 140" laat ook 130 door.
- Zeg eerst in een zin wanneer een afmeting **fout** is: *ze is geen 120, en ze is ook geen 140*. Let op het woordje **en**. Schrijf je "niet 120 **of** niet 140", dan is elke afmeting fout: 120 is niet 140, en 140 is niet 120. Je API weigert dan alles.
- Hoe je twee voorwaarden combineert, staat in je leerboek, hoofdstuk 4.
- Test daarom ook met de **goede** waarden. Een controle die alles weigert, zie je alleen als je ook eens 120 en 140 instuurt.

Stap 5. De prijs moet groter zijn dan 0

- "Groter dan 0" betekent dat **0 zelf ook fout** is. Het bericht zegt het eigenlijk al: 0 of lager wordt geweigerd.
- Test met 0 en met -5 (geweigerd), en met 0.01 (toegelaten).

Stap 6. De berichten, letterlijk

- Neem de drie berichten letterlijk over uit de opgave, met de punt achteraan.
- Is er meer dan één veld fout, dan geef je het bericht van de eerste controle in de volgorde van de opgave: merk, afmeting, prijs. Met de volgorde uit stap 1 gaat dat vanzelf: de eerste controle die faalt, stopt je methode.

Stap 7. Testen in Scalar

Pas telkens één waarde aan in de body, en vraag na elke 400 de volledige lijst op. Die moet even lang blijven.

Wat je aanpast in de body	Wat je moet krijgen	De lijst daarna
`"merk": ""`	400 met *Een ventilator moet een merk hebben.*	even lang
`"merk": "  "`	400 met hetzelfde bericht	even lang
`"afmetingInMm": 92`	400 met *We verkopen enkel ventilatoren van 120 of 140 mm.*	even lang
`"afmetingInMm": 130`	400 met hetzelfde bericht	even lang
`"afmetingInMm": 140`	201	één erbij
`"prijs": 0`	400 met *De prijs moet groter zijn dan 0.*	even lang
`"prijs": -5`	400 met hetzelfde bericht	even lang
`"prijs": 0.01`	201	één erbij
`"merk": ""` én `"afmetingInMm": 92`	400 met het bericht over het merk	even lang

Het voorbeeld dat Scalar zelf invult, met afmeting 0 en prijs 0, wordt vanaf nu geweigerd. Dat is juist.

Stap 8. Daarna: dezelfde controles in punt 5 en punt 8

De drie controles komen terug in je PUT en in de vervanging. Kopieer je ze drie keer, dan moet je een fout ook op drie plaatsen herstellen.

Krijg eerst de POST juist. Zet de controles daarna op één plaats in je controller, zodat de POST, de PUT en de vervanging ze kunnen gebruiken. Denk daarbij na wat die plaats aan je POST moet vertellen, zodat je POST weet óf hij moet weigeren, en met welk bericht.

Waar het staat

Wat	Waar
`BadRequest` in de tabel met statuscodes	les 02
`BadRequest` met een bericht, en de andere helpers	leerboek, hoofdstuk 26
een lege tekst nagaan	leerboek, hoofdstuk 1
`if` en twee voorwaarden combineren	leerboek, hoofdstuk 4
`null` en het vraagteken achter een type	leerboek, hoofdstuk 17

Zit je echt vast

Je oplossingen staan in `Oefeningen\Oplossingen Oefeningen\Oplossingen Hoofdstuk 2\Oplossing_02_02.md`, bij punt 4. Lees eerst alleen de nakijkpunten en de aanpak: die staan open. Probeer het daarna opnieuw zelf. Open pas daarna *Toon de code*, en vergelijk zoals in *Zo kijk je na*: doet mijn code hetzelfde?

Opkuis, zonder invloed op het antwoord

Waar	Wat	Richting
regel 24 en 37, tegenover 53 en 71	De parameter in je lambda heet bij de GET en de POST `fan`, bij de PUT en de DELETE `ventilator`.	Kies één schrijfwijze. De les gebruikt een korte letter, zoals `x`.
regel 71	De variabele `ventilator` en de parameter in de lambda heten hetzelfde, op dezelfde regel. Het compileert, maar bij het lezen zie je niet welke van de twee bedoeld is.	Geef de parameter in de lambda een andere naam.

Wat goed is, en waarom

- **Je lijst is nu `private static`, met een kleine letter**, precies zoals in de eindversie van les 02. De lijst hoort bij de klasse en blijft bestaan tussen twee verzoeken. Een tweede POST krijgt nu Id 6.
- **De twee overbodige `using`-regels zijn weg.** Ze stonden al in `GlobalUsings.cs`.
- **Je PUT zoekt eerst, en geeft een 404 mét `return`.** Dat is het patroon dat je in punt 4 nodig hebt. Je hebt het hier zelf al juist geschreven.
- **Je PUT neemt het Id niet over uit de body.** Het Id komt uit de URL, en de client kan het zo niet wijzigen. Elke andere eigenschap neem je wel over.
- `NoContent` **en** `ActionResult` **zonder type** bij de PUT en de DELETE. Er komt niets terug in de body, en het returntype zegt dat ook.
- **Je DELETE verwijdert het gevonden object** en geeft 204. Zodra A2 opgelost is, klopt hij volledig.
- **Je POST** haalt de ventilator uit de body, overschrijft een meegestuurd Id en geeft 201 met `CreatedAtAction` en `nameof`, zoals in de les.
- `Ventilator?` op regel 24, 53 en 71. Het vraagteken klopt telkens, want `FirstOrDefault` kan niets vinden.

Compilerwaarschuwingen

Voor zover ik kan nagaan door te lezen, geeft je code **één** waarschuwing: CS8604 op regel 78, bij A2. Ze verdwijnt zodra je `if` in de DELETE de methode stopt.

In Visual Studio zie je ze in de Error List, bij **Warnings**. Staat dat filter uit, dan zie je enkel de fouten.

Zelf nakijken

Doe na het oplossen deze verzoeken. Een antwoord klopt pas als de statuscode én het bericht letterlijk overeenkomen met de opgave.

Verzoek	Wat je moet zien	Hoort bij
`GET api/ventilator/99`	404 met *We vonden geen ventilator met Id 99.*	A3
de negen verzoeken uit stap 7	telkens het antwoord uit die tabel	A1 en punt 4
`PUT api/ventilator/99`	404 met *Kan geen ventilator bijwerken met Id 99, omdat deze niet bestaat.*	A3
`DELETE api/ventilator/2`, en nog eens	eerst 204, daarna 404 met *Kan ventilator met Id 2 niet verwijderen, omdat deze niet gevonden is.*	A2 en A3
bouwen met Ctrl+Shift+B	geen waarschuwing CS8604 meer	A2

Met het nakijkscript, als je API in Visual Studio gesloten is:

```
.\Nakijk.ps1 -Oefening 02_02 -Project
"...\ProgrammingAdvancedOefeningen\ProgrammingAdvancedOefeningen"
```

Het script stuurt alle verzoeken uit de opgave. Zolang punt 5 en punt 8 niet af zijn, staan die als fout in het verslag.

Vragen die je je misschien stelt

Waarom waarschuwt de compiler bij mijn DELETE, maar niet bij mijn POST? In je POST roep je `BadRequest` op zonder het resultaat te gebruiken. Dat is toegelaten, dus de compiler zegt niets. In je DELETE geef je een variabele die `null` kan zijn door aan `Remove`, en dat ziet de compiler wel. Het is dezelfde fout, maar alleen in de DELETE heeft ze een gevolg dat de compiler kan zien.

Ik stuur `"merk": null` en krijg een 400 in het Engels. Waar komt die vandaan? Van ASP.NET zelf. Je project staat op `Nullable` aan, en `Merk` is een `string` zonder vraagteken. ASP.NET behandelt het merk dan als verplicht, en `[ApiController]` weigert het verzoek nog voor je methode start. Je eigen bericht komt pas aan bod bij een lege tekst. De opgave spreekt over een leeg merk, dus daarop test je.

Moet ik de verlichting ook controleren? Nee. De opgave vraagt drie controles. Voeg er geen toe die niet gevraagd zijn: een test die een geldige ventilator stuurt, verwacht 201.

Waarom een 400 en geen 404 als het merk leeg is? Een 404 zegt: wat je zoekt, bestaat niet. Een 400 zegt: wat je stuurt, klopt niet. Bij een leeg merk zoekt de client niets, maar stuurt hij iets wat niet mag.

Wat als ik alle ventilatoren verwijder en dan een nieuwe toevoeg? Dan loopt je POST vast met een 500, omdat er in een lege lijst geen hoogste Id bestaat. De opgave vraagt hier niets over en de les doet het net zo, dus het is geen fout. Wil je het toch opvangen: je leerboek, hoofdstuk 11, zegt wat `Max` doet op een lege lijst.